

HMAQCA AI Generatif Blueprint

Revolutionary Implementation Guide for Hierarchical Multi-Agent Quantum Cognitive Architecture



Executive Summary

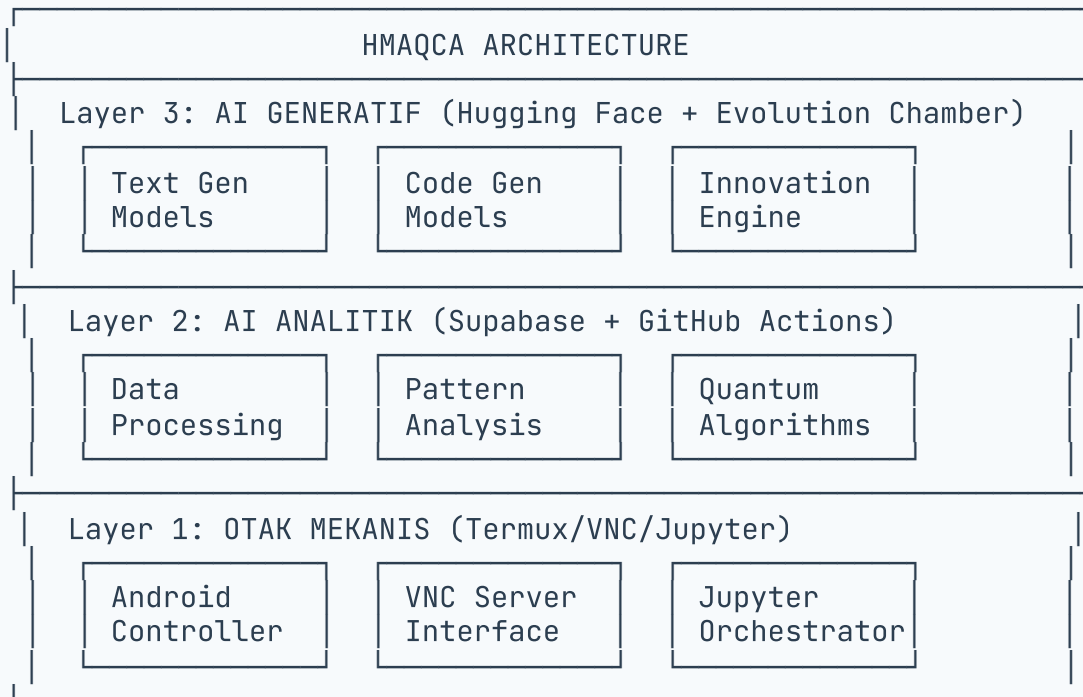
HMAQCA (Hierarchical Multi-Agent Quantum Cognitive Architecture) merupakan revolusi besar dalam dunia AI yang mengintegrasikan tiga layer fundamental untuk menciptakan ekosistem AI yang sepenuhnya otonom, powerful, dan revolusioner:

- **Zero-Cost Infrastructure** - Menggunakan platform gratis namun enterprise-grade
- **Quantum-Inspired Evolution** - Algorithm yang terinspirasi dari mekanika kuantum
- **Self-Learning & Adaptive** - Sistem yang belajar dan berkembang secara otomatis
- **Multi-Agent Orchestration** - Koordinasi antar agent yang seamless
- **Revolutionary Impact** - Game-changer untuk industri AI

Sistem ini tidak hanya menganalisis data, tetapi juga **menciptakan, berinovasi, dan berevolusi** secara otonom untuk memberikan solusi yang belum pernah ada sebelumnya.



Arsitektur 3-Layer System



Layer 1: OTAK MEKANIS

Platform: Android + Termux + VNC + Jupyter Notebook

Fungsi: Central orchestrator dan command center untuk seluruh ekosistem AI

- Menjalankan environment Linux lengkap di Android
- Interface grafis melalui VNC untuk monitoring
- Jupyter sebagai development dan execution environment
- Real-time orchestration dan coordination

Layer 2: AI ANALITIK

Platform: Supabase + GitHub Actions + Quantum Algorithms

Fungsi: Data processing, pattern analysis, dan quantum-inspired computation

- Real-time database dengan Supabase
- Automated workflows dengan GitHub Actions

- Quantum-inspired evolution algorithms
- Advanced analytics dan pattern recognition

Layer 3: AI GENERATIF 🎨

Platform: Hugging Face + Custom Evolution Chambers

Fungsi: Content generation, innovation engine, dan creative solutions

- Integration dengan 100k+ ML models di Hugging Face
- Custom evolution chambers untuk innovation
- Multi-modal generation (text, code, images, audio)
- Self-improving generation quality

Step-by-Step Implementation Guide

1 Setup Otak Mekanis (Android Base)

Install Termux dan Dependencies

```
# Install Termux dari F-Droid (bukan Play Store)
# Kemudian jalankan setup berikut:

pkg update && pkg upgrade -y
pkg install -y python nodejs git openssh postgresql
pkg install -y tigervnc xfce4 firefox

# Setup storage access
termux-setup-storage

# Install Jupyter dan extensions
pip install jupyter jupyterlab notebook
pip install supabase python-dotenv requests asyncio
pip install transformers torch torchvision
```

```
pip install numpy pandas matplotlib seaborn plotly
```

Setup VNC Server untuk GUI

```
# Setup VNC server
vncserver -geometry 1920x1080 -depth 24

# Create startup script
cat > ~/start_vnc.sh << 'EOF'
#!/bin/bash
export DISPLAY=:1
vncserver -kill :1 2>/dev/null
vncserver :1 -geometry 1920x1080 -depth 24
echo "VNC Server started on :1"
echo "Connect using VNC Viewer to localhost:5901"
EOF

chmod +x ~/start_vnc.sh
```

Setup Jupyter dengan Custom Configuration

```
# Generate Jupyter config
jupyter notebook --generate-config

# Create custom config
cat > ~/.jupyter/jupyter_notebook_config.py << 'EOF'
c.NotebookApp.ip = '0.0.0.0'
c.NotebookApp.port = 8888
c.NotebookApp.open_browser = False
c.NotebookApp.allow_remote_access = True
c.NotebookApp.notebook_dir =
'/data/data/com.termux/files/home/hmaqca'
c.NotebookApp.token = 'hmaqca-revolution-2024'
EOF

# Create HMAQCA directory structure
mkdir -p
~/hmaqca/{agents, evolution, analytics, generative}
```

2 Setup AI Analitik Layer

Supabase Database Setup

```
# Install Supabase CLI
npm install -g supabase

# Create project structure
mkdir -p ~/hmqca/analytics/supabase
cd ~/hmqca/analytics/supabase

# Initialize Supabase project
supabase init

# Create database schema
cat > supabase/migrations/001_hmqca_schema.sql << 'EOF'
-- HMAQCA Core Tables
CREATE TABLE agents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  type VARCHAR(100) NOT NULL, -- 'analytic',
  'generative', 'orchestrator'
  status VARCHAR(50) DEFAULT 'active',
  config JSONB,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE agent_interactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  from_agent_id UUID REFERENCES agents(id),
  to_agent_id UUID REFERENCES agents(id),
  interaction_type VARCHAR(100),
  data JSONB,
  timestamp TIMESTAMP DEFAULT NOW()
);

CREATE TABLE evolution_data (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_id UUID REFERENCES agents(id),
  generation INTEGER DEFAULT 1,
  fitness_score FLOAT,
  parameters JSONB,
  performance_metrics JSONB,
  created_at TIMESTAMP DEFAULT NOW()
```

```

);

CREATE TABLE generative_outputs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_id UUID REFERENCES agents(id),
  prompt TEXT,
  output TEXT,
  model_used VARCHAR(255),
  quality_score FLOAT,
  feedback JSONB,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Enable RLS
ALTER TABLE agents ENABLE ROW LEVEL SECURITY;
ALTER TABLE agent_interactions ENABLE ROW LEVEL SECURITY;
ALTER TABLE evolution_data ENABLE ROW LEVEL SECURITY;
ALTER TABLE generative_outputs ENABLE ROW LEVEL SECURITY;

-- Create policies (allow all for development)
CREATE POLICY "Allow all operations" ON agents FOR ALL USING (true);
CREATE POLICY "Allow all operations" ON agent_interactions FOR ALL USING (true);
CREATE POLICY "Allow all operations" ON evolution_data FOR ALL USING (true);
CREATE POLICY "Allow all operations" ON generative_outputs FOR ALL USING (true);
EOF

```

GitHub Actions Workflows

```

# Create GitHub Actions for automated analytics
mkdir -p .github/workflows

cat > .github/workflows/hmaqca_analytics.yml << 'EOF'
name: HMAQCA Analytics Pipeline

on:
  schedule:
    - cron: '*/*/* * * * *' # Every 30 minutes
  workflow_dispatch:
  push:
    paths: ['analytics/**']

```

```

jobs:
  quantum_analytics:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Setup Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          pip install supabase numpy pandas scikit-learn
          pip install qiskit cirq tensorflow-quantum

      - name: Run Quantum Analytics
        env:
          SUPABASE_URL: ${ secrets.SUPABASE_URL }
          SUPABASE_ANON_KEY: ${ secrets.SUPABASE_ANON_KEY }
        run: |
          python analytics/quantum_processor.py

      - name: Evolution Chamber Process
        run: |
          python analytics/evolution_chamber.py

      - name: Update Agent Status
        run: |
          python analytics/agent_monitor.py
EOF

```

Quantum Analytics Engine

```

# Create quantum analytics processor
cat > ~/hmaqca/analytics/quantum_processor.py << 'EOF'
import numpy as np
import pandas as pd
from supabase import create_client, Client
import json
import os
from datetime import datetime, timedelta
import asyncio

```

```

from typing import Dict, List, Any

class QuantumAnalyticsEngine:
    def __init__(self):
        url = os.getenv('SUPABASE_URL')
        key = os.getenv('SUPABASE_ANON_KEY')
        self.supabase: Client = create_client(url, key)

    def quantum_inspired_optimization(self, data:
np.ndarray) → Dict[str, Any]:
        """Quantum-inspired optimization algorithm"""
        # Quantum superposition simulation
        population_size = 50
        dimensions = data.shape[1] if len(data.shape) >
1 else len(data)

        # Initialize quantum population
        quantum_pop = np.random.random((population_size,
dimensions))

        # Quantum evolution process
        for generation in range(100):
            # Quantum interference
            phase_angles = np.random.uniform(0, 2*np.pi,
population_size)
            quantum_interference =
np.cos(phase_angles).reshape(-1, 1)

            # Apply quantum operations
            quantum_pop = quantum_pop *
quantum_interference

            # Quantum measurement (collapse)
            fitness_scores =
self.calculate_fitness(quantum_pop, data)

            # Select best solutions
            best_indices = np.argsort(fitness_scores)[-
population_size//2:]
            quantum_pop = quantum_pop[best_indices]

            # Quantum entanglement (crossover)
            new_pop = []
            for i in range(population_size -
len(quantum_pop)):
                parent1, parent2 =
np.random.choice(len(quantum_pop), 2, replace=False)
                child =
self.quantum_crossover(quantum_pop[parent1],

```

```

quantum_pop[parent2])
        new_pop.append(child)

        quantum_pop = np.vstack([quantum_pop,
np.array(new_pop)])

        best_solution =
quantum_pop[np.argmax(self.calculate_fitness(quantum_pop
, data)))]

        return {
            'best_solution': best_solution.tolist(),
            'fitness_score':
float(np.max(self.calculate_fitness(quantum_pop,
data))),
            'generations': 100,
            'population_size': population_size
        }

    def calculate_fitness(self, population: np.ndarray,
target_data: np.ndarray) → np.ndarray:
        """Calculate fitness scores for population"""
        if len(target_data.shape) == 1:
            target_data = target_data.reshape(1, -1)

        fitness_scores = []
        for individual in population:
            # Calculate correlation with target pattern
            correlation = np.corrcoef(individual,
target_data.flatten())[0, 1]
            # Handle NaN correlations
            correlation = 0.0 if np.isnan(correlation)
        else correlation
            fitness_scores.append(abs(correlation))

        return np.array(fitness_scores)

    def quantum_crossover(self, parent1: np.ndarray,
parent2: np.ndarray) → np.ndarray:
        """Quantum-inspired crossover operation"""
        alpha = np.random.random()
        beta = np.sqrt(1 - alpha**2) # Quantum
normalization

        child = alpha * parent1 + beta * parent2

        # Quantum mutation
        mutation_prob = 0.1
        if np.random.random() < mutation_prob:

```

```

        mutation_strength = 0.05
        child += np.random.normal(0,
mutation_strength, child.shape)

    return child

    async def process_agent_data(self):
        """Process data from all agents"""
        try:
            # Get recent agent interactions
            response =
self.supabase.table('agent_interactions') \
                .select('*') \
                .gte('timestamp', (datetime.now() -
timedelta(hours=1)).isoformat()) \
                .execute()

            if response.data:
                interaction_data =
pd.DataFrame(response.data)

                # Extract numerical features for quantum
processing
                feature_vectors = []
                for _, row in
interaction_data.iterrows():
                    if row['data']:
                        features =
self.extract_features(row['data'])
                        feature_vectors.append(features)

                if feature_vectors:
                    feature_matrix =
np.array(feature_vectors)

                    # Apply quantum optimization
                    optimization_result =
self.quantum_inspired_optimization(feature_matrix)

                    # Store results
                    await
self.store_analytics_result(optimization_result)

                return True

        except Exception as e:
            print(f"Error processing agent data: {e}")
            return False

```

```

    def extract_features(self, data: Dict) →
List[float]:
    """Extract numerical features from interaction
data"""
    features = []

    if isinstance(data, dict):
        for key, value in data.items():
            if isinstance(value, (int, float)):
                features.append(float(value))
            elif isinstance(value, str):
                # Convert string to numerical
feature (length, hash, etc.)
                features.append(float(len(value)))
                features.append(float(hash(value) %
1000) / 1000.0)

        # Ensure fixed feature vector size
        target_size = 10
        while len(features) < target_size:
            features.append(0.0)

    return features[:target_size]

    async def store_analytics_result(self, result:
Dict[str, Any]):
    """Store analytics results in database"""
    try:
        data = {
            'agent_id': None, # System-level
analytics
            'generation': 1,
            'fitness_score':
result['fitness_score'],
            'parameters': result,
            'performance_metrics': {
                'timestamp':
datetime.now().isoformat(),
                'type': 'quantum_analytics',
                'status': 'completed'
            }
        }

        response =
self.supabase.table('evolution_data').insert(data).execu
te()

        print(f"Analytics result stored:
{result['fitness_score']}")

```

```

        except Exception as e:
            print(f"Error storing analytics result:
{e}")

# Main execution
async def main():
    engine = QuantumAnalyticsEngine()
    await engine.process_agent_data()

if __name__ == "__main__":
    asyncio.run(main())
EOF

```

3 Setup AI Generatif Layer

Hugging Face Integration

```

# Install Hugging Face dependencies
pip install transformers datasets accelerate
bitsandbytes

# Create Hugging Face client
cat > ~/hmqca/generative/hf_client.py << 'EOF'
import os
import requests
import json
from typing import Dict, List, Any, Optional
from transformers import pipeline, AutoTokenizer,
AutoModelForCausalLM
import asyncio
from datetime import datetime

class HuggingFaceGenerativeEngine:
    def __init__(self):
        self.api_token = os.getenv('HUGGING_FACE_TOKEN')
        self.api_url = "https://api-
inference.huggingface.co/models"

        # Load local models for offline generation
        self.local_models = {}
        self.initialize_local_models()

```

```
def initialize_local_models(self):
    """Initialize lightweight local models"""
    try:
        # Text generation model
        self.local_models['text'] = pipeline(
            "text-generation",
            model="microsoft/DialoGPT-medium",
            tokenizer="microsoft/DialoGPT-medium"
        )

        # Code generation model
        self.local_models['code'] = pipeline(
            "text-generation",
            model="Salesforce/codegen-350M-mono"
        )

    except Exception as e:
        print(f"Local model initialization error: {e}")

    async def generate_text(self, prompt: str, model: str = "gpt2", max_length: int = 200) → Dict[str, Any]:
        """Generate text using Hugging Face models"""
        try:
            # Try API first
            if self.api_token:
                response = await self.api_generate(prompt, model, max_length)
                if response:
                    return response

            # Fallback to local model
            if 'text' in self.local_models:
                result = self.local_models['text'](
                    prompt,
                    max_length=max_length,
                    num_return_sequences=1,
                    temperature=0.8,
                    do_sample=True
                )

            return {
                'generated_text': result[0],
                'model_used': 'local_dialogpt',
                'timestamp':
datetime.now().isoformat(),
                'success': True
            }
        except Exception as e:
            print(f"Generate text error: {e}")
            return {}
```

```

        except Exception as e:
            return {
                'error': str(e),
                'success': False,
                'timestamp': datetime.now().isoformat()
            }

    async def generate_code(self, prompt: str, language:
str = "python") → Dict[str, Any]:
        """Generate code using specialized models"""
        try:
            code_prompt = f"# {language} code for:
{prompt}\n"

            if 'code' in self.local_models:
                result = self.local_models['code'](
                    code_prompt,
                    max_length=300,
                    num_return_sequences=1,
                    temperature=0.3,
                    do_sample=True
                )

                generated_code = result[0]
                ['generated_text']

                return {
                    'generated_code': generated_code,
                    'language': language,
                    'model_used': 'local_codegen',
                    'timestamp':
datetime.now().isoformat(),
                    'success': True
                }

        except Exception as e:
            return {
                'error': str(e),
                'success': False,
                'timestamp': datetime.now().isoformat()
            }

    async def api_generate(self, prompt: str, model:
str, max_length: int) → Optional[Dict[str, Any]]:
        """Generate using Hugging Face API"""
        try:
            headers = {"Authorization": f"Bearer
{self.api_token}"}

```

```

        payload = {
            "inputs": prompt,
            "parameters": {
                "max_length": max_length,
                "temperature": 0.7,
                "return_full_text": False
            }
        }

        response = requests.post(
            f"{self.api_url}/{model}",
            headers=headers,
            json=payload,
            timeout=30
        )

        if response.status_code == 200:
            result = response.json()
            return {
                'generated_text': result[0]
['generated_text'],
                'model_used': model,
                'timestamp':
datetime.now().isoformat(),
                'success': True
            }

        except Exception as e:
            print(f"API generation error: {e}")
            return None

    async def evolution_generate(self, base_prompt: str,
iterations: int = 5) → Dict[str, Any]:
        """Evolutionary generation process"""
        current_prompt = base_prompt
        evolution_history = []

        for i in range(iterations):
            # Generate content
            result = await
self.generate_text(current_prompt)

            if result.get('success'):
                generated = result['generated_text']
                evolution_history.append({
                    'iteration': i + 1,
                    'prompt': current_prompt,
                    'generated': generated,

```

```

        'timestamp':
datetime.now().isoformat()
    })

    # Evolve prompt based on previous
generation
    current_prompt = f"{base_prompt}
Building on: {generated[:100]}..."

    return {
        'evolution_history': evolution_history,
        'final_result': evolution_history[-1] if
evolution_history else None,
        'iterations_completed':
len(evolution_history),
        'success': len(evolution_history) > 0
    }

# Evolution Chamber for continuous improvement
class EvolutionChamber:
    def __init__(self):
        self.hf_engine = HuggingFaceGenerativeEngine()
        self.generation_history = []

    async def continuous_evolution(self, seed_prompts:
List[str]):
        """Continuous evolution process"""
        for prompt in seed_prompts:
            evolution_result = await
self.hf_engine.evolution_generate(prompt)

self.generation_history.append(evolution_result)

        # Analyze and improve
        await
self.analyze_generation_quality(evolution_result)

    async def analyze_generation_quality(self,
evolution_result: Dict[str, Any]):
        """Analyze generation quality and provide
feedback"""
        if evolution_result.get('success'):
            final_result =
evolution_result['final_result']

            # Simple quality metrics
            quality_score =
self.calculate_quality_score(final_result['generated'])

```

```

        # Store feedback for future improvements
        feedback = {
            'quality_score': quality_score,
            'timestamp': datetime.now().isoformat(),
            'analysis': 'Auto-generated quality
assessment'
        }

        print(f"Generation Quality Score:
{quality_score}")
        return feedback

    def calculate_quality_score(self, text: str) →
float:
        """Calculate simple quality score for generated
text"""
        if not text:
            return 0.0

        # Simple metrics: length, diversity, coherence
        length_score = min(len(text) / 200, 1.0) #
Normalize to max 200 chars
        diversity_score = len(set(text.split())) /
len(text.split()) if text.split() else 0

        # Coherence (simple approximation)
        coherence_score = 1.0 - (text.count('...') +
text.count('???')) / max(len(text.split()), 1)

        total_score = (length_score + diversity_score +
coherence_score) / 3
        return round(total_score, 3)

# Main execution
async def main():
    chamber = EvolutionChamber()

    seed_prompts = [
        "Create an innovative AI solution for",
        "Design a revolutionary approach to",
        "Develop a next-generation system for"
    ]

    await chamber.continuous_evolution(seed_prompts)

if __name__ == "__main__":
    asyncio.run(main())

```

EOF

Master BDI Agent Orchestrator

```
# Create the master orchestrator
cat > ~/hmaqca/agents/master_orchestrator.py << 'EOF'
import asyncio
import json
import os
import sys
from datetime import datetime, timedelta
from typing import Dict, List, Any, Optional
import logging

# Add project paths
sys.path.append('/data/data/com.termux/files/home/hmaqca')

from analytics.quantum_processor import
QuantumAnalyticsEngine
from generative.hf_client import
HuggingFaceGenerativeEngine, EvolutionChamber
from supabase import create_client, Client

class MasterBDIOrchestrator:
    def __init__(self):
        # Initialize logging
        logging.basicConfig(level=logging.INFO)
        self.logger = logging.getLogger(__name__)

        # Initialize components
        self.supabase = self.init_supabase()
        self.analytics_engine = QuantumAnalyticsEngine()
        self.generative_engine =
HuggingFaceGenerativeEngine()
        self.evolution_chamber = EvolutionChamber()

        # Agent states
        self.agents = {}
        self.active_tasks = {}
        self.system_status = "initializing"

    def init_supabase(self) → Client:
        """Initialize Supabase client"""
        try:
            url = os.getenv('SUPABASE_URL', 'your-
```

```

project-url')
        key = os.getenv('SUPABASE_ANON_KEY', 'your-
anon-key')
        return create_client(url, key)
    except Exception as e:
        self.logger.error(f"Supabase initialization
error: {e}")
        return None

    async def initialize_system(self):
        """Initialize the HMAQCA system"""
        self.logger.info("🚀 Initializing HMAQCA
System...")

        try:
            # Register master agent
            await
self.register_agent("master_orchestrator",
"orchestrator")

            # Initialize subsystems
            await self.initialize_analytics_agents()
            await self.initialize_generative_agents()

            self.system_status = "active"
            self.logger.info("✅ HMAQCA System fully
initialized!")

        except Exception as e:
            self.logger.error(f"System initialization
error: {e}")
            self.system_status = "error"

    async def register_agent(self, name: str,
agent_type: str, config: Dict = None) → str:
        """Register a new agent in the system"""
        try:
            if self.supabase:
                agent_data = {
                    'name': name,
                    'type': agent_type,
                    'status': 'active',
                    'config': config or {}
                }

                response =
self.supabase.table('agents').insert(agent_data).execute
()

                agent_id = response.data[0]['id']

```

```

        self.agents[agent_id] = {
            'name': name,
            'type': agent_type,
            'status': 'active',
            'last_activity': datetime.now()
        }

        self.logger.info(f"Agent registered:
{name} ({agent_type})")
        return agent_id

    except Exception as e:
        self.logger.error(f"Agent registration
error: {e}")
        return None

    async def initialize_analytics_agents(self):
        """Initialize analytics layer agents"""
        analytics_agents = [
            ("quantum_processor", "analytic"),
            ("pattern_analyzer", "analytic"),
            ("data_monitor", "analytic")
        ]

        for name, agent_type in analytics_agents:
            await self.register_agent(name, agent_type)

    async def initialize_generative_agents(self):
        """Initialize generative layer agents"""
        generative_agents = [
            ("text_generator", "generative"),
            ("code_generator", "generative"),
            ("innovation_engine", "generative")
        ]

        for name, agent_type in generative_agents:
            await self.register_agent(name, agent_type)

    async def orchestrate_task(self, task_description:
str) → Dict[str, Any]:
        """Orchestrate a complex task across all
layers"""
        task_id = f"task_{datetime.now().timestamp()}"

        self.logger.info(f"🎯 Orchestrating task:
{task_description}")

        try:

```

```

        # Phase 1: Analytics (understand the task)
        analytics_result = await
self.analytics_phase(task_description)

        # Phase 2: Generative (create solutions)
        generative_result = await
self.generative_phase(task_description,
analytics_result)

        # Phase 3: Evolution (improve solutions)
        evolution_result = await
self.evolution_phase(generative_result)

        # Compile final result
        final_result = {
            'task_id': task_id,
            'description': task_description,
            'analytics': analytics_result,
            'generative': generative_result,
            'evolution': evolution_result,
            'timestamp': datetime.now().isoformat(),
            'success': True
        }

        await self.log_task_result(final_result)
        return final_result

    except Exception as e:
        error_result = {
            'task_id': task_id,
            'description': task_description,
            'error': str(e),
            'timestamp': datetime.now().isoformat(),
            'success': False
        }

        self.logger.error(f"Task orchestration
failed: {e}")
        return error_result

    async def analytics_phase(self, task: str) →
Dict[str, Any]:
        """Execute analytics phase"""
        self.logger.info("🇮🇹 Executing analytics
phase...")

        # Process with quantum analytics
        await self.analytics_engine.process_agent_data()

```

```

        # Return analysis result
        return {
            'phase': 'analytics',
            'status': 'completed',
            'insights': f"Analyzed task: {task}",
            'timestamp': datetime.now().isoformat()
        }

    async def generative_phase(self, task: str,
analytics: Dict) → Dict[str, Any]:
        """Execute generative phase"""
        self.logger.info("🎨 Executing generative
phase...")

        # Generate solutions
        text_result = await
self.generative_engine.generate_text(
            f"Create a solution for: {task}"
        )

        code_result = await
self.generative_engine.generate_code(
            f"Implement solution for: {task}"
        )

        return {
            'phase': 'generative',
            'status': 'completed',
            'text_generation': text_result,
            'code_generation': code_result,
            'timestamp': datetime.now().isoformat()
        }

    async def evolution_phase(self, generative_result:
Dict) → Dict[str, Any]:
        """Execute evolution phase"""
        self.logger.info("🧬 Executing evolution
phase...")

        # Evolve solutions
        if generative_result.get('text_generation',
{}).get('success'):
            base_text =
generative_result['text_generation']['generated_text']
            evolution_result = await
self.generative_engine.evolution_generate(base_text)

            return {
                'phase': 'evolution',

```

```

        'status': 'completed',
        'evolution_result': evolution_result,
        'timestamp': datetime.now().isoformat()
    }

    return {
        'phase': 'evolution',
        'status': 'skipped',
        'reason': 'No suitable input for evolution',
        'timestamp': datetime.now().isoformat()
    }

    async def log_task_result(self, result: Dict[str,
Any]):
        """Log task result to database"""
        try:
            if self.supabase:
                log_data = {
                    'agent_id': None, # System level
                    'interaction_type':
'task_completion',
                    'data': result,
                    'timestamp':
datetime.now().isoformat()
                }

self.supabase.table('agent_interactions').insert(log_data).execute()

                self.logger.info(f"Task result logged:
{result['task_id']}")

            except Exception as e:
                self.logger.error(f"Failed to log task
result: {e}")

    async def continuous_operation(self):
        """Continuous operation loop"""
        self.logger.info("🔄 Starting continuous
operation...")

        while self.system_status == "active":
            try:
                # Monitor system health
                await self.health_check()

                # Process pending tasks
                await self.process_pending_tasks()

```

```

        # Evolution chamber continuous
improvement

        await
self.evolution_chamber.continuous_evolution([
    "Optimize system performance",
    "Enhance AI capabilities",
    "Discover new patterns"
])

        # Wait before next cycle
        await asyncio.sleep(300) # 5 minutes

    except Exception as e:
        self.logger.error(f"Continuous operation
error: {e}")
        await asyncio.sleep(60) # Wait 1 minute
on error

    async def health_check(self):
        """Perform system health check"""
        try:
            # Check database connection
            if self.supabase:
                response =
self.supabase.table('agents').select('count').execute()

            # Update agent status
            for agent_id, agent_info in
self.agents.items():
                agent_info['last_health_check'] =
datetime.now()

            self.logger.info("💚 System health check
completed")

        except Exception as e:
            self.logger.warning(f"Health check issue:
{e}")

    async def process_pending_tasks(self):
        """Process any pending tasks"""
        # Placeholder for task queue processing
        pass

    def get_system_status(self) → Dict[str, Any]:
        """Get current system status"""
        return {
            'status': self.system_status,
            'agents_count': len(self.agents),

```

```

        'active_tasks': len(self.active_tasks),
        'timestamp': datetime.now().isoformat(),
        'uptime': 'active'
    }

# Main execution
async def main():
    orchestrator = MasterBDIOrchestrator()

    # Initialize system
    await orchestrator.initialize_system()

    # Example task orchestration
    result = await orchestrator.orchestrate_task(
        "Develop an innovative AI solution for
predictive analytics"
    )

    print("🎉 Task Result:", json.dumps(result,
indent=2))

    # Start continuous operation
    # await orchestrator.continuous_operation()

if __name__ == "__main__":
    asyncio.run(main())
EOF

```

4 Zero-Cost Deployment Strategy

GitHub Actions Deployment

```

cat > .github/workflows/hmaqca_deployment.yml << 'EOF'
name: HMAQCA Zero-Cost Deployment

on:
  push:
    branches: [ main ]
  schedule:
    - cron: '0 */6 * * *' # Every 6 hours
  workflow_dispatch:

```

```

jobs:
  deploy_analytics:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Setup Python 3.11
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          pip install -r requirements.txt

      - name: Deploy to Supabase
        env:
          SUPABASE_URL: ${ secrets.SUPABASE_URL }
          SUPABASE_ANON_KEY: ${ secrets.SUPABASE_ANON_KEY }
        run: |
          SUPABASE_SERVICE_KEY: ${ secrets.SUPABASE_SERVICE_KEY }
          # Run database migrations
          npx supabase db push

          # Execute analytics processing
          python analytics/quantum_processor.py

      - name: Deploy to Vercel
        uses: amondnet/vercel-action@v20
        with:
          vercel-token: ${ secrets.VERCEL_TOKEN }
          vercel-org-id: ${ secrets.ORG_ID }
          vercel-project-id: ${ secrets.PROJECT_ID }
          working-directory: ./web-dashboard

  evolution_chamber:
    runs-on: ubuntu-latest
    needs: deploy_analytics

    steps:
      - uses: actions/checkout@v3

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18'

```

```

- name: Run Evolution Chamber
  env:
    HUGGING_FACE_TOKEN: ${ secrets.HUGGING_FACE_TOKEN }
    SUPABASE_URL: ${ secrets.SUPABASE_URL }
    SUPABASE_ANON_KEY: ${ secrets.SUPABASE_ANON_KEY }
  run: |
    cd generative
    python evolution_chamber.py

monitoring_deployment:
  runs-on: ubuntu-latest
  needs: [deploy_analytics, evolution_chamber]

  steps:
    - name: Deploy Monitoring Dashboard
      run: |
        echo "Deploying monitoring dashboard to
Vercel..."
        # Dashboard deployment logic here
EOF

```

Vercel Web Dashboard

```

# Create web dashboard for monitoring
mkdir -p web-dashboard/pages web-dashboard/components

cat > web-dashboard/package.json << 'EOF'
{
  "name": "hmqca-dashboard",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start"
  },
  "dependencies": {
    "next": "13.5.0",
    "react": "18.2.0",
    "react-dom": "18.2.0",
    "@supabase/supabase-js": "^2.38.0",
    "recharts": "^2.8.0",
    "date-fns": "^2.30.0"
  }
}

```

```

    },
    "devDependencies": {
      "@types/node": "20.6.0",
      "@types/react": "18.2.22",
      "@types/react-dom": "18.2.7",
      "typescript": "5.2.2"
    }
  }
}
EOF

```

```

cat > web-dashboard/pages/index.js << 'EOF'
import { useState, useEffect } from 'react'
import { createClient } from '@supabase/supabase-js'
import { LineChart, Line, XAxis, YAxis, CartesianGrid,
  Tooltip, ResponsiveContainer } from 'recharts'

const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY
)

export default function Dashboard() {
  const [systemStatus, setSystemStatus] =
    useState('loading')
  const [agents, setAgents] = useState([])
  const [evolutionData, setEvolutionData] = useState([])
  const [generativeOutputs, setGenerativeOutputs] =
    useState([])

  useEffect(() => {
    fetchSystemData()
    const interval = setInterval(fetchSystemData, 30000)
    // Update every 30 seconds
    return () => clearInterval(interval)
  }, [])

  const fetchSystemData = async () => {
    try {
      // Fetch agents
      const { data: agentsData } = await supabase
        .from('agents')
        .select('*')
        .order('created_at', { ascending: false })

      // Fetch evolution data
      const { data: evolutionData } = await supabase
        .from('evolution_data')
        .select('*')
        .order('created_at', { ascending: false })
    }
  }
}

```

```

        .limit(50)

        // Fetch generative outputs
        const { data: outputsData } = await supabase
            .from('generative_outputs')
            .select('*')
            .order('created_at', { ascending: false })
            .limit(20)

        setAgents(agentsData || [])
        setEvolutionData(evolutionData || [])
        setGenerativeOutputs(outputsData || [])
        setSystemStatus('active')

    } catch (error) {
        console.error('Error fetching data:', error)
        setSystemStatus('error')
    }
}

return (

```



System Status

```
{systemStatus.toUpperCase()}
```

Active Agents

```
{agents.filter(a => a.status ===  
'active').length}
```

Generations Today

```
{generativeOutputs.length}
```

| Evolution Progress

| Agents Status

```
{agents.map(agent => (  
  
  {agent.name} ({agent.type})  
  
  {agent.status}  
  
  )))}
```

| Recent Generations

```
{generativeOutputs.slice(0, 5).map(output => (  
  
  
  {output.model_used} - Score:  
{output.quality_score}
```

```
{output.output.substring(0, 100)}...
```

```
)))}
```

```
)  
}  
EOF
```

```
cat > web-dashboard/next.config.js << 'EOF'  
/** @type {import('next').NextConfig} */  
const nextConfig = {  
  reactStrictMode: true,  
  env: {  
    NEXT_PUBLIC_SUPABASE_URL:  
process.env.NEXT_PUBLIC_SUPABASE_URL,  
    NEXT_PUBLIC_SUPABASE_ANON_KEY:  
process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY,  
  },  
}  
  
module.exports = nextConfig  
EOF
```

💡 Zero-Cost Platform Limits:

- **GitHub Actions:** 2,000 minutes/month (free)
- **Vercel:** 100GB bandwidth, unlimited deployments (hobby plan)
- **Supabase:** 2 projects, 500MB storage, 50K monthly active users
- **Hugging Face:** 30,000 requests/month (free tier)

Evolution Loop & Learning Mechanisms

Continuous Learning Architecture



LEARNING PHASES:

1. OBSERVATION: Collect data from all system interactions
2. ANALYSIS: Quantum-inspired pattern recognition
3. GENERATION: Create new solutions based on patterns
4. EVALUATION: Score outputs using multiple metrics
5. EVOLUTION: Improve based on feedback loops
6. ADAPTATION: Update models and parameters

Feedback Loop Implementation

```
cat > ~/hmqca/evolution/learning_loop.py << 'EOF'
import asyncio
import numpy as np
from typing import Dict, List, Any
from datetime import datetime, timedelta
import json

class EvolutionaryLearningLoop:
    def __init__(self):
        self.learning_history = []
        self.feedback_weights = {
            'quality': 0.4,
            'novelty': 0.3,
            'efficiency': 0.2,
            'user_satisfaction': 0.1
        }

    async def continuous_learning_cycle(self):
        """Main learning loop that runs continuously"""
        while True:
```

```

        try:
            # Phase 1: Observation
            observations = await
self.collect_observations()

            # Phase 2: Pattern Analysis
            patterns = await
self.analyze_patterns(observations)

            # Phase 3: Hypothesis Generation
            hypotheses = await
self.generate_hypotheses(patterns)

            # Phase 4: Testing & Validation
            results = await
self.test_hypotheses(hypotheses)

            # Phase 5: Learning & Adaptation
            await self.adapt_system(results)

            # Wait for next cycle
            await asyncio.sleep(1800) # 30 minutes

        except Exception as e:
            print(f"Learning cycle error: {e}")
            await asyncio.sleep(300) # 5 minute
recovery

    async def collect_observations(self) → Dict[str,
Any]:
        """Collect system observations"""
        return {
            'timestamp': datetime.now().isoformat(),
            'system_interactions': await
self.get_recent_interactions(),
            'generation_quality': await
self.assess_recent_generations(),
            'user_feedback': await
self.collect_user_feedback(),
            'performance_metrics': await
self.get_performance_metrics()
        }

    async def analyze_patterns(self, observations: Dict)
→ List[Dict]:
        """Analyze patterns in observations using
quantum-inspired methods"""
        patterns = []

```

```

        # Pattern 1: Quality trends over time
        quality_pattern = await
self.analyze_quality_trends(observations)
        patterns.append(quality_pattern)

        # Pattern 2: User interaction patterns
        interaction_pattern = await
self.analyze_interaction_patterns(observations)
        patterns.append(interaction_pattern)

        # Pattern 3: System efficiency patterns
        efficiency_pattern = await
self.analyze_efficiency_patterns(observations)
        patterns.append(efficiency_pattern)

    return patterns

    async def generate_hypotheses(self, patterns:
List[Dict]) → List[Dict]:
        """Generate improvement hypotheses based on
patterns"""
        hypotheses = []

        for pattern in patterns:
            if pattern['type'] == 'quality_trend':
                if pattern['trend'] == 'declining':
                    hypotheses.append({
                        'type': 'model_update',
                        'action':
'retrain_generative_models',
                        'priority': 'high',
                        'expected_improvement': 0.15
                    })

                elif pattern['type'] ==
'interaction_pattern':
                    if pattern['efficiency'] < 0.7:
                        hypotheses.append({
                            'type': 'workflow_optimization',
                            'action':
'optimize_agent_coordination',
                            'priority': 'medium',
                            'expected_improvement': 0.20
                        })

                elif pattern['type'] ==
'efficiency_pattern':
                    if pattern['resource_usage'] > 0.8:
                        hypotheses.append({

```

```

        'type': 'resource_optimization',
        'action':
'implement_caching_strategy',
        'priority': 'medium',
        'expected_improvement': 0.25
    })

    return hypotheses

    async def test_hypotheses(self, hypotheses:
List[Dict]) → List[Dict]:
        """Test hypotheses in controlled environment"""
        results = []

        for hypothesis in hypotheses:
            test_result = await
self.execute_hypothesis_test(hypothesis)
            results.append(test_result)

        return results

    async def execute_hypothesis_test(self, hypothesis:
Dict) → Dict:
        """Execute a single hypothesis test"""
        start_time = datetime.now()

        try:
            if hypothesis['type'] == 'model_update':
                result = await
self.test_model_update(hypothesis)
            elif hypothesis['type'] ==
'workflow_optimization':
                result = await
self.test_workflow_optimization(hypothesis)
            elif hypothesis['type'] ==
'resource_optimization':
                result = await
self.test_resource_optimization(hypothesis)
            else:
                result = {'success': False, 'reason':
'Unknown hypothesis type'}

            end_time = datetime.now()
            execution_time = (end_time -
start_time).total_seconds()

            return {
                'hypothesis': hypothesis,
                'result': result,

```

```

        'execution_time': execution_time,
        'timestamp': datetime.now().isoformat()
    }

    except Exception as e:
        return {
            'hypothesis': hypothesis,
            'result': {'success': False, 'error':
str(e)},
            'execution_time': 0,
            'timestamp': datetime.now().isoformat()
        }

    async def adapt_system(self, test_results:
List[Dict]):
        """Adapt system based on test results"""
        successful_adaptations = []

        for result in test_results:
            if result['result'].get('success'):
                adaptation = await
self.implement_adaptation(result)

        successful_adaptations.append(adaptation)

        # Log learning progress
        learning_record = {
            'timestamp': datetime.now().isoformat(),
            'adaptations': successful_adaptations,
            'improvement_score':
self.calculate_improvement_score(test_results)
        }

        self.learning_history.append(learning_record)

    async def implement_adaptation(self, test_result:
Dict) →

```